
Modul 3: Architecture və Design Patterns

Gün 61-90 - arxitektura stilləri, pattern-lər, test strategiyası və final capstone

Bu paket backend developer üçün server əsasları, sistem dizaynı, arxitektura, dizayn pattern-lər, performans, etibarlılıq və production düşüncəsini ardıcıl öyrənmək üçün hazırlanıb. Hər gün bir mövzu oxu, həmin günün praktik tapşırığını yaz, sonra öz sistem dizayn qeydlərini yenilə.

Təvsiyə olunan gündəlik ritm: 60-90 dəqiqə nəzəriyyə, 60-120 dəqiqə praktika, 20 dəqiqə review.

Modulun istifadə qaydası

1. Hər günün əvvəlində məqsədi oxu və həmin mövzunu öz sözlərinlə izah etməyə çalış.
2. Nəzəriyyəni oxuduqdan sonra praktikanı mütləq kodla və ya dizayn sənədi ilə tamamla.
3. Mini-lab hissəsində ölçmə, test və ya failure scenario yarat. Production düşüncəsi burada formalaşır.
4. Müsahibə suallarına cavabı səsli izah et. Cavabın strukturunu sadə saxla: problem, seçim, trade-off, risk.

Gün 61: Arxitektura qərarları və ADR yazmaq

Öyrənmə məqsədi

Günün sonunda 'Arxitektura qərarları və ADR yazmaq' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Arxitektura qərarları və ADR yazmaq real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 62: Layered architecture və dependency direction

Öyrənmə məqsədi

Günün sonunda 'Layered architecture və dependency direction' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Layered architecture və dependency direction real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 63: Clean/Hexagonal architecture

Öyrənmə məqsədi

Günün sonunda 'Clean/Hexagonal architecture' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Clean/Hexagonal architecture real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 64: Domain-driven design əsasları

Öyrənmə məqsədi

Günün sonunda 'Domain-driven design əsasları' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Domain-driven design əsasları real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 65: Aggregate, entity, value object və invariant

Öyrənmə məqsədi

Günün sonunda 'Aggregate, entity, value object və invariant' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Aggregate, entity, value object və invariant real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 66: Repository və Unit of Work pattern

Öyrənmə məqsədi

Günün sonunda 'Repository və Unit of Work pattern' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Repository və Unit of Work pattern real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 67: Factory, Builder və Object Mother

Öyrənmə məqsədi

Günün sonunda 'Factory, Builder və Object Mother' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Factory, Builder və Object Mother real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 68: Strategy və Policy pattern

Öyrənmə məqsədi

Günün sonunda 'Strategy və Policy pattern' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Strategy və Policy pattern real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 69: Adapter, Facade və Anti-corruption Layer

Öyrənmə məqsədi

Günün sonunda 'Adapter, Facade və Anti-corruption Layer' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Adapter, Facade və Anti-corruption Layer real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 70: Decorator, Proxy və middleware chain

Öyrənmə məqsədi

Günün sonunda 'Decorator, Proxy və middleware chain' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Decorator, Proxy və middleware chain real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 71: Command, CQRS və query model

Öyrənmə məqsədi

Günün sonunda 'Command, CQRS və query model' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Command, CQRS və query model real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 72: Saga və process manager

Öyrənmə məqsədi

Günün sonunda 'Saga və process manager' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Saga və process manager real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 73: Event sourcing nə vaxt uyğundur

Öyrənmə məqsədi

Günün sonunda 'Event sourcing nə vaxt uyğundur' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Design pattern-lər təkrarlanan dizayn problemlərinə ad verilmiş həllərdir. Onları əzbərləmək yox, nə vaxt lazımsız complexity yaratdığını və nə vaxt coupling-i azaltdığını anlamaq vacibdir. Backend-də Repository, Unit of Work, Strategy, Adapter, Factory, Decorator, Command, Saga və Outbox çox praktikdir.

Praktiki tətbiq

Bir payment provider integrasiyasını Adapter ilə ayır. Stripe və mock provider eyni interface-i implement etsin.

Mini-lab

Discount hesablaması üçün Strategy pattern qur. Percentage, fixed amount və coupon-based strategiyaları əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Event sourcing nə vaxt uyğundur real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 74: Modular monolith dizaynı

Öyrənmə məqsədi

Günün sonunda 'Modular monolith dizaynı' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Modular monolith dizaynı real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 75: Microservices: sərhədlər və trade-off-lar

Öyrənmə məqsədi

Günün sonunda 'Microservices: sərhədlər və trade-off-lar' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Microservices: sərhədlər və trade-off-lar real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 76: Event-driven architecture

Öyrənmə məqsədi

Günün sonunda 'Event-driven architecture' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Event-driven architecture real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 77: API gateway və BFF pattern

Öyrənmə məqsədi

Günün sonunda 'API gateway və BFF pattern' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. API gateway və BFF pattern real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 78: Testing pyramid və backend test strategiyası

Öyrənmə məqsədi

Günün sonunda 'Testing pyramid və backend test strategiyası' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Backend test strategiyası confidence və sürət balansıdır. Unit test business rule-u tez yoxlayır, integration test real dependency-lərlə contract-ı təsdiqləyir, end-to-end test kritik user flow-u qoruyur. Test datanın idarəsi və flaky test-lərin azaldılması production keyfiyyətinə birbaşa təsir edir.

Praktiki tətbiq

Bir endpoint üçün unit, integration və contract test nümunələri yaz. Hər testin hansı riski tutduğunu qeyd et.

Mini-lab

Database-li testləri isolated transaction və ya container ilə işə sal. Test order-independence yoxlaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Testing pyramid və backend test strategiyası real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 79: Unit, integration və contract test

Öyrənmə məqsədi

Günün sonunda 'Unit, integration və contract test' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Backend test strategiyası confidence və sürət balansıdır. Unit test business rule-u tez yoxlayır, integration test real dependency-lərlə contract-ı təsdiqləyir, end-to-end test kritik user flow-u qoruyur. Test datanın idarəsi və flaky test-lərin azaldılması production keyfiyyətinə birbaşa təsir edir.

Praktiki tətbiq

Bir endpoint üçün unit, integration və contract test nümunələri yaz. Hər testin hansı riski tutduğunu qeyd et.

Mini-lab

Database-li testləri isolated transaction və ya container ilə işə sal. Test order-independence yoxlaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Unit, integration və contract test real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 80: Testcontainers və local dependency testing

Öyrənmə məqsədi

Günün sonunda 'Testcontainers və local dependency testing' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Backend test strategiyası confidence və sürət balansıdır. Unit test business rule-u tez yoxlayır, integration test real dependency-lərlə contract-ı təsdiqləyir, end-to-end test kritik user flow-u qoruyur. Test datanın idarəsi və flaky test-lərin azaldılması production keyfiyyətinə birbaşa təsir edir.

Praktiki tətbiq

Bir endpoint üçün unit, integration və contract test nümunələri yaz. Hər testin hansı riski tutduğunu qeyd et.

Mini-lab

Database-li testləri isolated transaction və ya container ilə işə sal. Test order-independence yoxlaması apar.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Testcontainers və local dependency testing real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 81: CI/CD pipeline və quality gate

Öyrənmə məqsədi

Günün sonunda 'CI/CD pipeline və quality gate' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Operations düşüncəsi kodun deploydan sonra necə yaşayacağını anlamaqdır. Config, secrets, logs, metrics, health checks, release strategy və rollback planı backend-in bir hissəsidir. Yaxşı deploy strategiyası istifadəçiyə təsiri minimuma endirir və komandaya sürətli bərpa imkanı verir.

Praktiki tətbiq

Servis üçün Dockerfile, healthcheck və environment config siyahısı yaz. Staging və production fərqlərini ayır.

Mini-lab

Canary deployment ssenarisi qur: 5 faiz traffic yeni versiyaya getsin, error rate artarsa rollback qərarı ver.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. CI/CD pipeline və quality gate real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 82: Code review üçün arxitektura checklist

Öyrənmə məqsədi

Günün sonunda 'Code review üçün arxitektura checklist' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Code review üçün arxitektura checklist real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 83: Refactoring: legacy kodu təhlükəsiz dəyişmək

Öyrənmə məqsədi

Günün sonunda 'Refactoring: legacy kodu təhlükəsiz dəyişmək' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Refactoring: legacy kodu təhlükəsiz dəyişmək real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 84: Technical debt və prioritization

Öyrənmə məqsədi

Günün sonunda 'Technical debt və prioritization' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Arxitektura kodun dəyişiklik qarşısında davranışdır. Modul sərhədləri, dependency direction, domain language, deployment unit və data ownership qərarları uzunmüddətli maintainability yaradır. Pattern məqsəd deyil; konkret ağrını azaltmaq üçün istifadə olunan alətdir.

Praktiki tətbiq

Mövcud və ya sadə CRUD layihəni module-lara böl: api, application, domain, infrastructure. Hər modulun nəyi bilməyə icazəsi olduğunu yaz.

Mini-lab

Bir business rule-u controller-dən çıxarıb domain service və ya aggregate metoduna yerləşdir. Testlə invariant-ı qoruduğunu göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Technical debt və prioritization real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 85: Incident postmortem və on-call hazırlığı

Öyrənmə məqsədi

Günün sonunda 'Incident postmortem və on-call hazırlığı' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Operations düşüncəsi kodun deploydan sonra necə yaşayacağını anlamaqdır. Config, secrets, logs, metrics, health checks, release strategy və rollback planı backend-in bir hissəsidir. Yaxşı deploy strategiyası istifadəçiyə təsiri minimuma endirir və komandaya sürətli bərpa imkanı verir.

Praktiki tətbiq

Servis üçün Dockerfile, healthcheck və environment config siyahısı yaz. Staging və production fərqlərini ayır.

Mini-lab

Canary deployment ssenarisi qur: 5 faiz traffic yeni versiyaya getsin, error rate artarsa rollback qərarı ver.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Incident postmortem və on-call hazırlığı real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 86: System design case: payment + notification

Öyrənmə məqsədi

Günün sonunda 'System design case: payment + notification' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. System design case: payment + notification real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 87: System design case: feed + search

Öyrənmə məqsədi

Günün sonunda 'System design case: feed + search' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. System design case: feed + search real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 88: System design case: booking + consistency

Öyrənmə məqsədi

Günün sonunda 'System design case: booking + consistency' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. System design case: booking + consistency real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 89: Portfolio layihəsi və README yazmaq

Öyrənmə məqsədi

Günün sonunda 'Portfolio layihəsi və README yazmaq' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Portfolio layihəsi və README yazmaq real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 90: Final mock interview və review planı

Öyrənmə məqsədi

Günün sonunda 'Final mock interview və review planı' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Final mock interview və review planı real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.